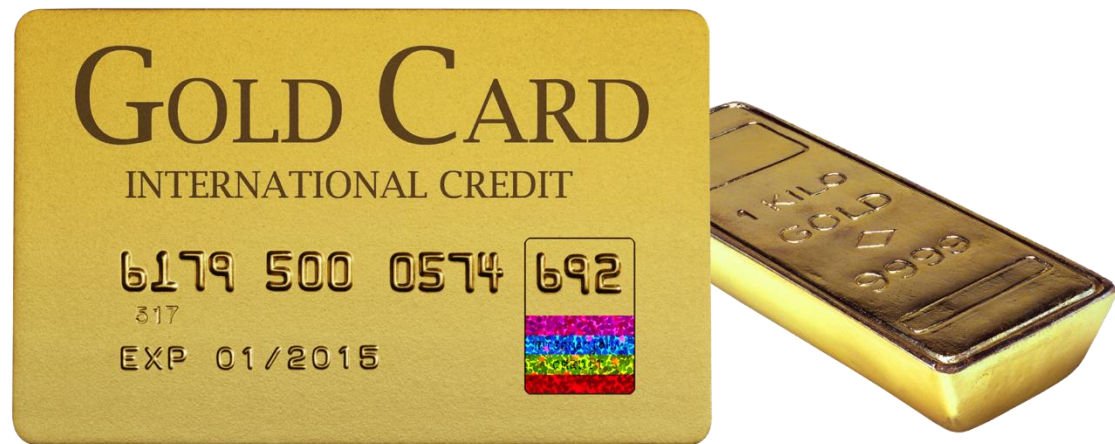


# Object-Oriented Programming



# Terminology

- ❑ Each **object** created in a program is an **instance** of a **class**.
- ❑ Each class presents to the outside world a concise and consistent view of the objects that are instances of this class, without going into too much unnecessary detail or giving others access to the inner workings of the objects.
- ❑ The class definition typically specifies **instance variables**, also known as **data members**, that the object contains, as well as the **methods**, also known as **member functions**, that the object can execute.

# Goals

## □ Robustness

- We want software to be capable of handling unexpected inputs that are not explicitly defined for its application.

## □ Adaptability

- Software needs to be able to evolve over time in response to changing conditions in its environment.

## □ Reusability

- The same code should be usable as a component of different systems in various applications.

# Object-Oriented Design Principles

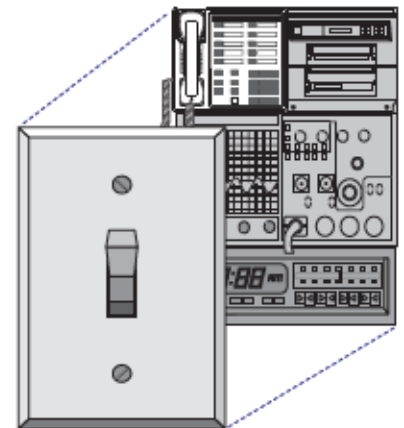
- ❑ Modularity
- ❑ Abstraction
- ❑ Encapsulation



Modularity



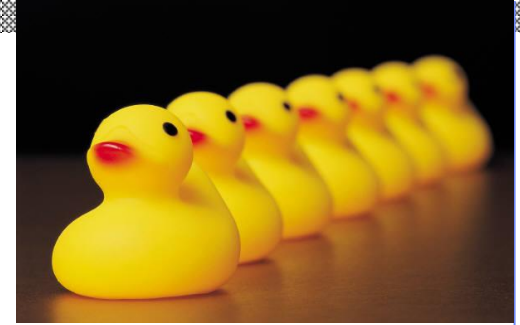
Abstraction



Encapsulation

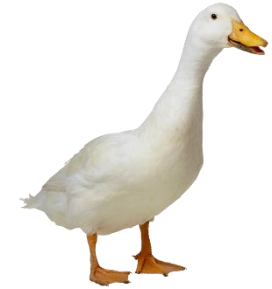
# Abstract Data Types

- ❑ **Abstraction** is to distill a system to its most fundamental parts.
- ❑ Applying the abstraction paradigm to the design of data structures gives rise to **abstract data types** (ADTs).
- ❑ An ADT is a model of a data structure that specifies the **type** of data stored, the **operations** supported on them, and the types of parameters of the operations.
- ❑ An ADT specifies what each operation does, but not how it does it.
- ❑ The collective set of behaviors supported by an ADT is its **public interface**.



# Duck Typing

- ❑ Python treats abstractions implicitly using a mechanism known as **duck typing**.
  - A program can treat objects as having certain functionality and they will behave correctly provided those objects provide this expected functionality.
- ❑ As an interpreted and dynamically typed language, there is no “compile time” checking of data types in Python, and no formal requirement for declarations of abstract base classes.
- ❑ The term “duck typing” comes from an adage attributed to poet James Whitcomb Riley, stating that “when I see a bird that walks like a duck and swims like a duck and quacks like a duck, I call that bird a duck.”



# Abstract Base Classes

- ❑ Python supports abstract data types using a mechanism known as an **abstract base class (ABC)**.
- ❑ An abstract base class cannot be instantiated, but it defines one or more common methods that all implementations of the abstraction must have.
- ❑ An ABC is realized by one or more concrete classes that inherit from the abstract base class while providing implementations for those methods declared by the ABC.
- ❑ We can make use of several existing abstract base classes coming from Python's collections module, which includes definitions for several common data structure ADTs, and concrete implementations of some of these.

# Encapsulation

- Another important principle of object-oriented design is **encapsulation**.
  - Different components of a software system should not reveal the internal details of their respective implementations.
- Some aspects of a data structure are assumed to be public and some others are intended to be internal details.
- Python provides only loose support for encapsulation.
  - By convention, names of members of a class (both data members and member functions) that start with a single underscore character (e.g., `_secret`) are assumed to be nonpublic and should not be relied upon.



# Design Patterns



## ❑ **Algorithmic patterns:**

- ❑ Recursion
- ❑ Amortization
- ❑ Divide-and-conquer
- ❑ Prune-and-search
- ❑ Brute force
- ❑ Dynamic programming
- ❑ The greedy method

## ❑ **Software design patterns:**

- ❑ Iterator
- ❑ Adapter
- ❑ Position
- ❑ Composition
- ❑ Template method
- ❑ Locator
- ❑ Factory method

# Object-Oriented SW Design

- ❑ **Responsibilities:** Divide the work into different actors, each with a different responsibility.
- ❑ **Independence:** Define the work for each class to be as independent from other classes as possible.
- ❑ **Behaviors:** Define the behaviors for each class carefully and precisely, so that the consequences of each action performed by a class will be well understood by other classes that interact with it.

# Unified Modeling Language (UML)

A **class diagram** has three portions.

1. The name of the class
2. The recommended instance variables
3. The recommended methods of the class.

|            |                                                                                                                           |                                                                                      |
|------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Class:     | CreditCard                                                                                                                |                                                                                      |
| Fields:    | <code>_customer</code><br><code>_bank</code><br><code>_account</code>                                                     | <code>_balance</code><br><code>_limit</code>                                         |
| Behaviors: | <code>get_customer()</code><br><code>get_bank()</code><br><code>get_account()</code><br><code>make_payment(amount)</code> | <code>get_balance()</code><br><code>get_limit()</code><br><code>charge(price)</code> |

# Class Definitions

- ❑ A class serves as the primary means for abstraction in object-oriented programming.
- ❑ In Python, every piece of data is represented as an instance of some class.
- ❑ A class provides a set of behaviors in the form of member functions (also known as **methods**), with implementations that belong to all its instances.
- ❑ A class also serves as a blueprint for its instances, effectively determining the way that state information for each instance is represented in the form of **attributes** (also known as **fields**, **instance variables**, or **data members**).

# The **self** Identifier

- ❑ In Python, the **self** identifier plays a key role.
- ❑ In any class, there can possibly be many different instances, and each must maintain its own instance variables.
- ❑ Therefore, each instance stores its own instance variables to reflect its current state. Syntactically, **self** identifies the instance upon which a method is invoked.

# Example

```
1 class CreditCard:
2     """ A consumer credit card."""
3
4     def __init__(self, customer, bank, acct, limit):
5         """ Create a new credit card instance.
6
7         The initial balance is zero.
8
9         customer the name of the customer (e.g., 'John Bowman')
10        bank      the name of the bank (e.g., 'California Savings')
11        acct      the account identifier (e.g., '5391 0375 9387 5309')
12        limit     credit limit (measured in dollars)
13        """
14        self._customer = customer
15        self._bank = bank
16        self._account = acct
17        self._limit = limit
18        self._balance = 0
19
```

# Example, Part 2

```
20  def get_customer(self):
21      """Return name of the customer."""
22      return self._customer
23
24  def get_bank(self):
25      """Return the bank's name."""
26      return self._bank
27
28  def get_account(self):
29      """Return the card identifying number (typically stored as a string)."""
30      return self._account
31
32  def get_limit(self):
33      """Return current credit limit."""
34      return self._limit
35
36  def get_balance(self):
37      """Return current balance."""
38      return self._balance
```

# Example, Part 3

```
39  def charge(self, price):
40      """ Charge given price to the card, assuming sufficient credit limit.
41
42      Return True if charge was processed; False if charge was denied.
43      """
44      if price + self._balance > self._limit:      # if charge would exceed limit,
45          return False                             # cannot accept charge
46      else:
47          self._balance += price
48          return True
49
50  def make_payment(self, amount):
51      """ Process customer payment that reduces balance. """
52      self._balance -= amount
```



# Constructors

- ❑ A user can create an instance of the CreditCard class using a syntax as:

```
cc = CreditCard('John Doe', '1st Bank', '5391 0375 9387 5309', 1000)
```

- ❑ Internally, this results in a call to the specially named `__init__` method that serves as the constructor of the class.
- ❑ Its primary responsibility is to establish the state of a newly created credit card object with appropriate instance variables.

# Operator Overloading

- ❑ Python's built-in classes provide natural semantics for many operators.
- ❑ For example, the syntax `a + b` invokes addition for numeric types, yet concatenation for sequence types.
- ❑ When defining a new class, we must consider whether a syntax like `a + b` should be defined when `a` or `b` is an instance of that class.

# Iterators

- ❑ Iteration is an important concept in the design of data structures.
- ❑ An **iterator** for a collection provides one key behavior:
  - It supports a special method named `__next__` that returns the next element of the collection, if any, or raises a `StopIteration` exception to indicate that there are no further elements.

# Automatic Iterators

- Python also helps by providing an automatic iterator implementation for any class that defines both `__len__` and `__getitem__`.

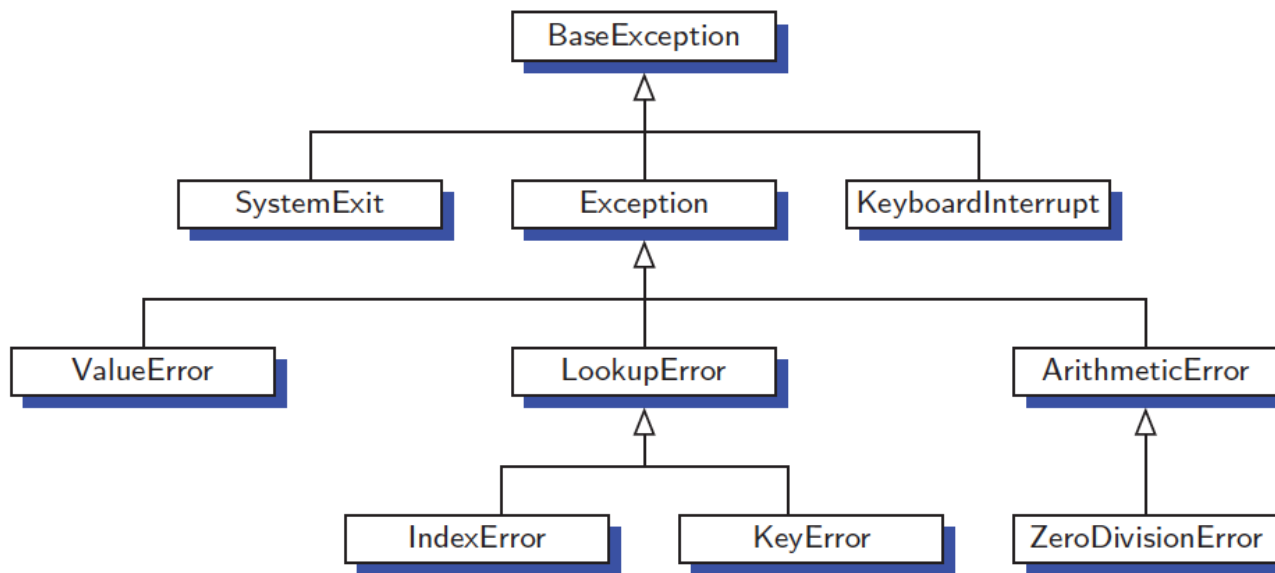
```
1 class Range:
2     """A class that mimics the built-in range class."""
3
4     def __init__(self, start, stop=None, step=1):
5         """Initialize a Range instance.
6
7         Semantics is similar to built-in range class.
8         """
9         if step == 0:
10             raise ValueError('step cannot be 0')
11
12         if stop is None:                # special case of range(n)
13             start, stop = 0, start      # should be treated as if range(0,n)
14
15         # calculate the effective length once
16         self._length = max(0, (stop - start + step - 1) // step)
17
18         # need knowledge of start and step (but not stop) to support __getitem__
19         self._start = start
20         self._step = step
21
22     def __len__(self):
23         """Return number of entries in the range."""
24         return self._length
25
26     def __getitem__(self, k):
27         """Return entry at index k (using standard interpretation if negative)."""
28         if k < 0:
29             k += len(self)             # attempt to convert negative index
30
31         if not 0 <= k < self._length:
32             raise IndexError('index out of range')
33
34         return self._start + k * self._step
```

# Inheritance

- ❑ A mechanism for a modular and hierarchical organization is **inheritance**.
- ❑ This allows a new class to be defined based upon an existing class as the starting point.
- ❑ The existing class is typically described as the **base class**, parent class, or superclass, while the newly defined class is known as the **subclass** or child class.
- ❑ There are two ways in which a subclass can differentiate itself from its superclass:
  - A subclass may specialize an existing behavior by providing a new implementation that overrides an existing method.
  - A subclass may also extend its superclass by providing brand new methods.

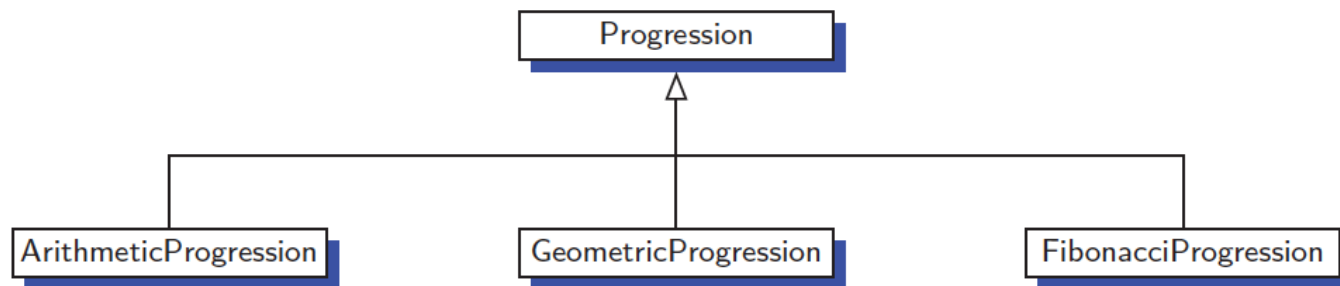
# Inheritance is Built into Python

- A portion of Python's hierarchy of exception types:



# An Extended Example

- A **numeric progression** is a sequence of numbers, where each number depends on one or more of the previous numbers.
  - An **arithmetic progression** determines the next number by adding a fixed constant to the previous value.
  - A **geometric progression** determines the next number by multiplying the previous value by a fixed constant.
  - A **Fibonacci progression** uses the formula  $N_{i+1} = N_i + N_{i-1}$



# The Progression Base Class

```
1 class Progression:
2     """Iterator producing a generic progression.
3
4     Default iterator produces the whole numbers 0, 1, 2, ...
5     """
6
7     def __init__(self, start=0):
8         """Initialize current to the first value of the progression."""
9         self._current = start
10
11     def _advance(self):
12         """Update self._current to a new value.
13
14         This should be overridden by a subclass to customize progression.
15
16         By convention, if current is set to None, this designates the
17         end of a finite progression.
18         """
19         self._current += 1
20
21     def __next__(self):
22         """Return the next element, or else raise StopIteration error."""
23         if self._current is None:      # our convention to end a progression
24             raise StopIteration()
25         else:
26             answer = self._current      # record current value to return
27             self._advance()             # advance to prepare for next time
28             return answer               # return the answer
29
30     def __iter__(self):
31         """By convention, an iterator must return itself as an iterator."""
32         return self
33
34     def print_progression(self, n):
35         """Print next n values of the progression."""
36         print(' '.join(str(next(self)) for j in range(n)))
```



# ArithmeticProgression Subclass

```
1 class ArithmeticProgression(Progression):           # inherit from Progression
2     """Iterator producing an arithmetic progression."""
3
4     def __init__(self, increment=1, start=0):
5         """Create a new arithmetic progression.
6
7         increment    the fixed constant to add to each term (default 1)
8         start        the first term of the progression (default 0)
9         """
10        super().__init__(start)                       # initialize base class
11        self._increment = increment
12
13    def _advance(self):                                # override inherited version
14        """Update current value by adding the fixed increment."""
15        self._current += self._increment
```

# GeometricProgression Subclass

```
1 class GeometricProgression(Progression):           # inherit from Progression
2     """Iterator producing a geometric progression."""
3
4     def __init__(self, base=2, start=1):
5         """Create a new geometric progression.
6
7         base        the fixed constant multiplied to each term (default 2)
8         start       the first term of the progression (default 1)
9         """
10        super().__init__(start)
11        self._base = base
12
13    def _advance(self):                               # override inherited version
14        """Update current value by multiplying it by the base value."""
15        self._current *= self._base
```

# FibonacciProgression Subclass

```
1 class FibonacciProgression(Progression):
2     """Iterator producing a generalized Fibonacci progression."""
3
4     def __init__(self, first=0, second=1):
5         """Create a new fibonacci progression.
6
7         first        the first term of the progression (default 0)
8         second       the second term of the progression (default 1)
9         """
10        super().__init__(first)           # start progression at first
11        self._prev = second - first       # fictitious value preceding the first
12
13    def _advance(self):
14        """Update current value by taking sum of previous two."""
15        self._prev, self._current = self._current, self._prev + self._current
```